

Simulating Blackjack Game using Java programming

Tuan Tran

Whitting school of Engineering

John Hopkins University

EN.605.201 - Introduction to programming with java

October 28th, 2025

Abstract

This paper summarizes the key design pattern of a java program that simulates the Blackjack cards game. The project uses an object-oriented programming design that demonstrates key characteristics of java including inheritance and polymorphism. The program includes 6 main classes including cards, decks, players, BlackjackPlayer, Dealer, Game, and the main class that runs the simulation. While object deck and object card have a composition relationship, Blackjack Player and Dealer followed by a parent-child design pattern and game uses all of the classes. Throughout the project, key relationships and interactions between those classes will be described and analyzed in conjunction with a discussion around other alternative design patterns.

General Program Design

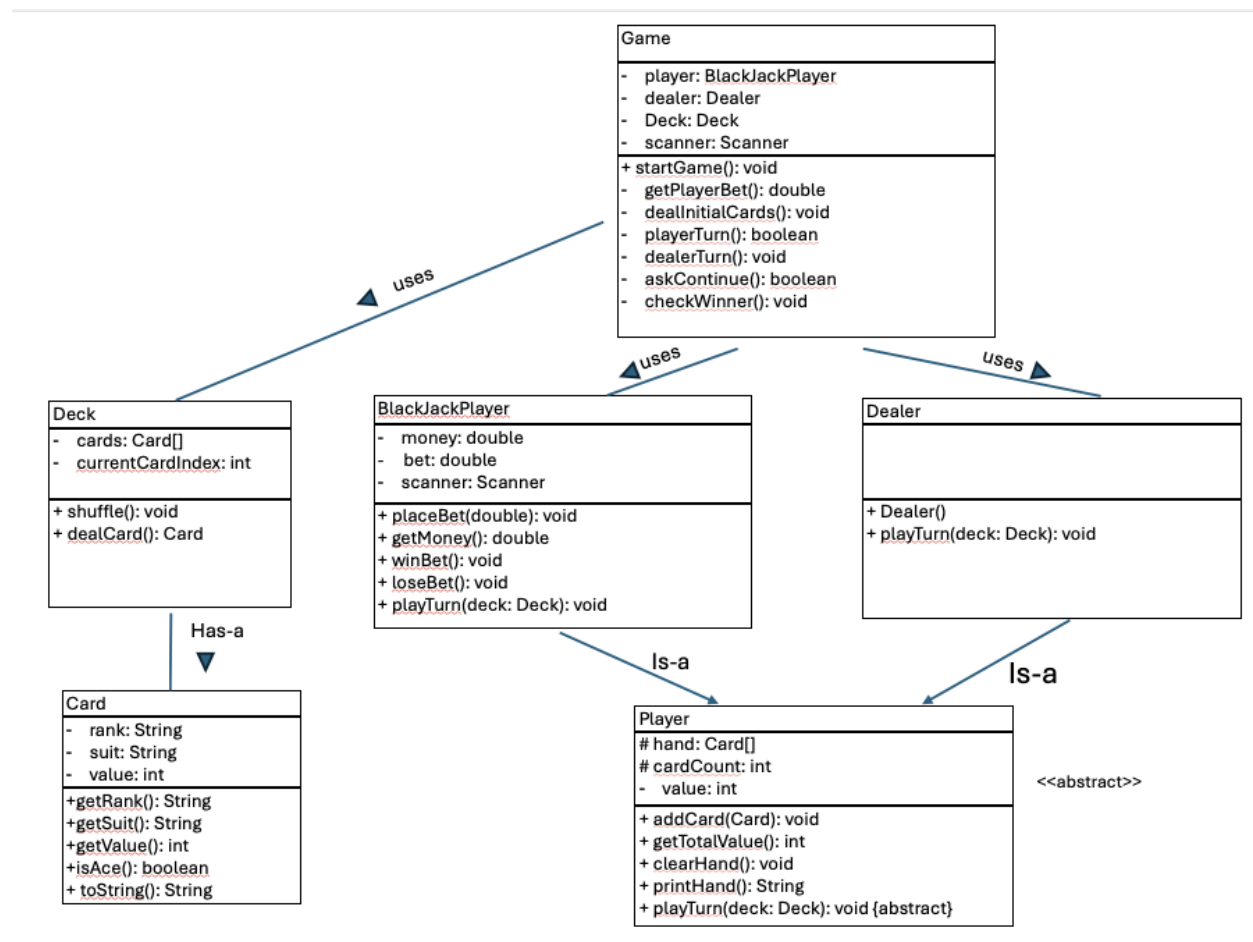
Important associations between classes

Intuitively, this Blackjack game includes 3 big objects, a Dealer, a Player and a Deck comprised of 52 cards. First, card class represents an individual playing cards with 3 key attributes: rank, suit, and value in the Blackjack game. Object card is designed such that we can retrieve the rank, suit and value of the card in the game. Ace is the most unique card since it can be either 1 or 11 depending on whichever value is most beneficial to the player. Thus, besides the three getter methods, an isAce method is included to be handled in such scenario

Since a deck has multiple cards (52 specially in this case), the relationship between object Deck and object Card is a “has-a” relationship. To be specific, Deck is an array of cards. Two main tasks of a deck is to deal cards and to shuffle cards hence the 2 methods are included in this card. While shuffle return void, method dealCard is used to get the next card from the deck

There are two types of players in this game: Dealer and the player who's betting their money. Essentially, two parties will have to obey the same rules. Therefore, the abstract class of Player is designed as a abstract class to construct similar traits between a Dealer and a BlackjackPlayer. Abstract class Player provides methods for adding cards, calculating the total value, clearing the hand, printing out the cards and defines an abstract method for playing a turn, indicating that Dealer and BlackjackPlayer class must implement this method in their own specific way. While Dealer Class inherits all the construct method from its abstract class – Player and only implement the playTurn method in line with the rule of Blackjack game, BlackjackPlayer also has an initial money and the ability to place bets. This means that BlacjackPlayer can win or lose their bet, and those two methods are also added to calculate the total money of the player.

The game class is the controller of the entire game and encapsulating the game flow. In Game class, it will include a player, a dealer, a deck, and a scanner for input. It started the game, handles player bets, deal cards, manage turns for player and dealer and check for winner according to Blackjack rules



Alternative Approaches Considered

Another pattern design that could have been implemented is the command pattern as opposed to the existing solutions, which simply simulate 2 parties involved and a deck of cards. In my current design pattern, I'm basically hardcoding the `playTurn` logic inside the **BlackjackPlayer** and **Dealer** classes and letting the game class control the entire game. We could have designed an interface that represents actions that can be performed during the Blackjack

game, then other commands such as dealing cards, hit or stay encapsulated with specific game action. However, the current design is straightforward and directly focuses on the key stakeholders and objects of the game without extensive abstraction or collection of commands. The proposed design introduced some abstractions and more command classes which could enhance the project extensively and make it slightly more complex to execute. In short, simplicity and clarity are paramount in this design, and its benefits could outweigh the proposed design pattern

Reference

- Doe, J. (2020, March 1). How to code blackjack using JavaScript. That Software Dude. <https://www.thatsoftwaredude.com/content/6417/how-to-code-blackjack-using-javascript>
- Brilliant. (n.d.). Programming Blackjack. Brilliant.org. <https://brilliant.org/wiki/programming-blackjack/>